

E-Commerce Product Entity-Resolution & Matching Platform

React.js · Spring Boot · PostgreSQL · Java

Bhargava · 2520090169

Hanish · 2520090107

SYSTEM OVERVIEW

- Full-stack web application identifying and deduplicating product listings across e-commerce datasets
- React.js dashboard for CSV upload, pipeline monitoring, and cluster exploration
- Spring Boot backend housing the hand-implemented algorithm engine exposed via REST APIs
- PostgreSQL persistence for products, candidate pairs, clusters, and export results

ALGORITHM ENGINE

KMP / Rabin-Karp	Pattern search and blocking
Wagner-Fischer DP	Fuzzy title matching
Suffix Array + LCP	Candidate retrieval index
Jaccard Similarity	Token overlap scoring
Union-Find DSU	Entity cluster merging
Miller-Rabin	Double-hash prime selection

INTRODUCTION

SCALE

Millions of product listings across Amazon, Flipkart and Walmart

PROBLEM

Same product, different sellers - inconsistent titles, descriptions, and attributes

SOLUTION

Automated Entity Resolution pipeline via classical DSA algorithms in a full-stack web platform

PROBLEM STATEMENT

- Manual deduplication is infeasible at the scale of modern e-commerce platforms
- Entity Resolution (ER) is the process of identifying and linking records referring to the same real-world product
- This project automates ER through an algorithm pipeline served over REST APIs and visualised in a React dashboard

MOTIVATION

- Demonstrate that classical algorithms solve production-grade data quality problems without ML
- Deliver a complete full-stack product: algorithm engine, REST API layer, interactive frontend, and database persistence
- REST-based design allows the engine to integrate with any external catalogue or marketplace system

SOFTWARE & HARDWARE REQUIREMENTS

SOFTWARE

COMPONENT	SPECIFICATION
Frontend	React.js, Tailwind CSS
Backend	Java 17+, Spring Boot
Database	PostgreSQL 15+
Build Tools	Maven, npm
IDE	VS Code
Input	CSV product dataset files
Output	Resolved cluster JSON + CSV export
API	RESTful JSON over HTTP

HARDWARE

COMPONENT	SPECIFICATION
RAM	8 GB recommended
Processor	Quad-core or above
Storage	1 GB disk space
OS	Windows / Linux / macOS
JVM	Java 17+ compliant runtime
Architecture	x86_64 / ARM compatible
Network	Localhost or cloud deployable

SYSTEM ARCHITECTURE

FRONTEND	CSV Upload	Pipeline Dashboard	Cluster Explorer	Product Comparison View
----------	------------	--------------------	------------------	-------------------------

▼ HTTP REST / JSON

BACKEND	Ingestion Controller	Preprocessing Service	Algorithm Engine	Cluster Service	Export Controller
---------	----------------------	-----------------------	------------------	-----------------	-------------------

▼ JPA / JDBC

DATABASE	Products Table	Candidates Table	Clusters Table	Results Table
----------	----------------	------------------	----------------	---------------

Algorithm Engine (Spring Boot service layer) — KMP · Rabin-Karp · Suffix Array · Wagner-Fischer DP · Union-Find DSU · Miller-Rabin — implemented in pure Java

YEAR	AUTHOR	CONCEPT	CONTRIBUTION TO THIS PROJECT
1966	Levenshtein	Edit Distance	Fuzzy product title matching via Wagner-Fischer DP; foundational to similarity scoring
1977	Knuth-Morris-Pratt	Linear Pattern Matching	$O(n+m)$ title search; eliminates redundant comparisons in the blocking stage
1987	Rabin-Karp	Rolling Polynomial Hash	Multi-pattern search and collision-resistant double-hashing in candidate retrieval
1997	Gusfield	Suffix Arrays and LCP	Substring similarity queries powering the inverted index blocking structure
2012	Christen	Record Linkage & Entity Resolution	Blocking strategy reducing $O(n^2)$ comparison space; grounds the full ER pipeline design

1 Data Ingestion

CSV uploaded via React UI → REST API → parsed and stored in PostgreSQL Products table with ID, title, description, category, price

2 Preprocessing

Custom Trie tokenizer splits titles and descriptions · HashMap removes stop-words · All tokens lowercased and normalized

3 Candidate Blocking

Inverted Index maps tokens to product IDs · Suffix Array and LCP retrieves substring-sharing pairs · Collapses $O(n^2)$ comparisons to a candidate set

4 Similarity Scoring

Wagner-Fischer Levenshtein DP scores title pairs · Jaccard Similarity scores token set overlap · Combined into a weighted similarity score per candidate pair

5 Graph Clustering

Products modelled as graph nodes · Pairs above similarity threshold become edges · Min-Heap priority queue ranks by confidence · Union-Find DSU merges into resolved entity clusters

6 Hash Safety

Rabin-Karp rolling hash uses Miller-Rabin primality testing to select large safe primes · Ensures collision-resistant double-hashing in blocking index

7 Output and Export

Clusters persisted to PostgreSQL · Visualised in React cluster explorer · Exportable as CSV · Available via REST endpoint for external consumption

Thank You

Bhargava · 2520090169

Hanish · 2520090107

React.js · Spring Boot · PostgreSQL · Java · DSA